



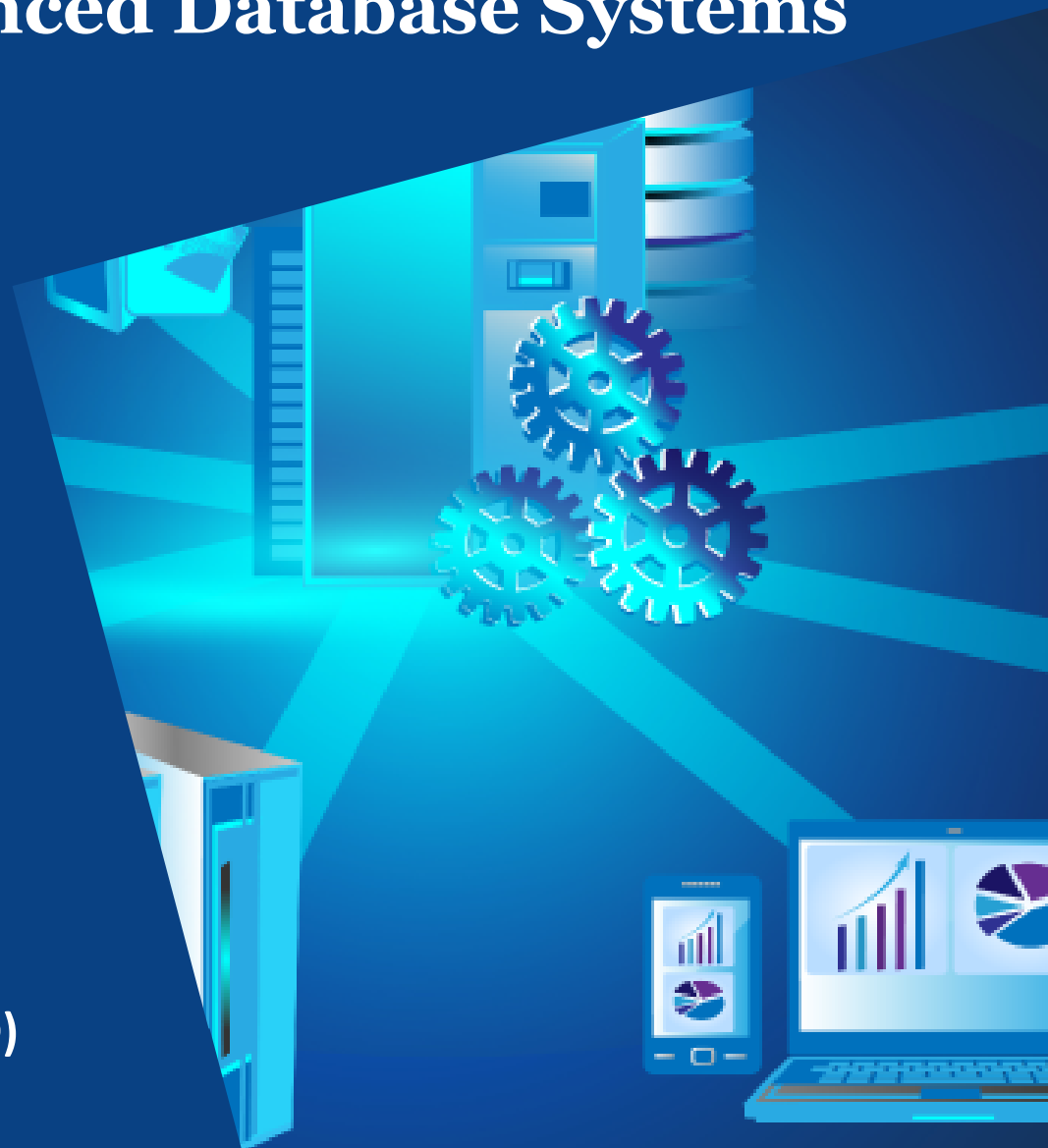
THE UNIVERSITY OF
MELBOURNE

COMP90050 Advanced Database Systems

Semester 1, 2026

Lecturer: Farhana Choudhury (PhD)

Live lecture – Week 5

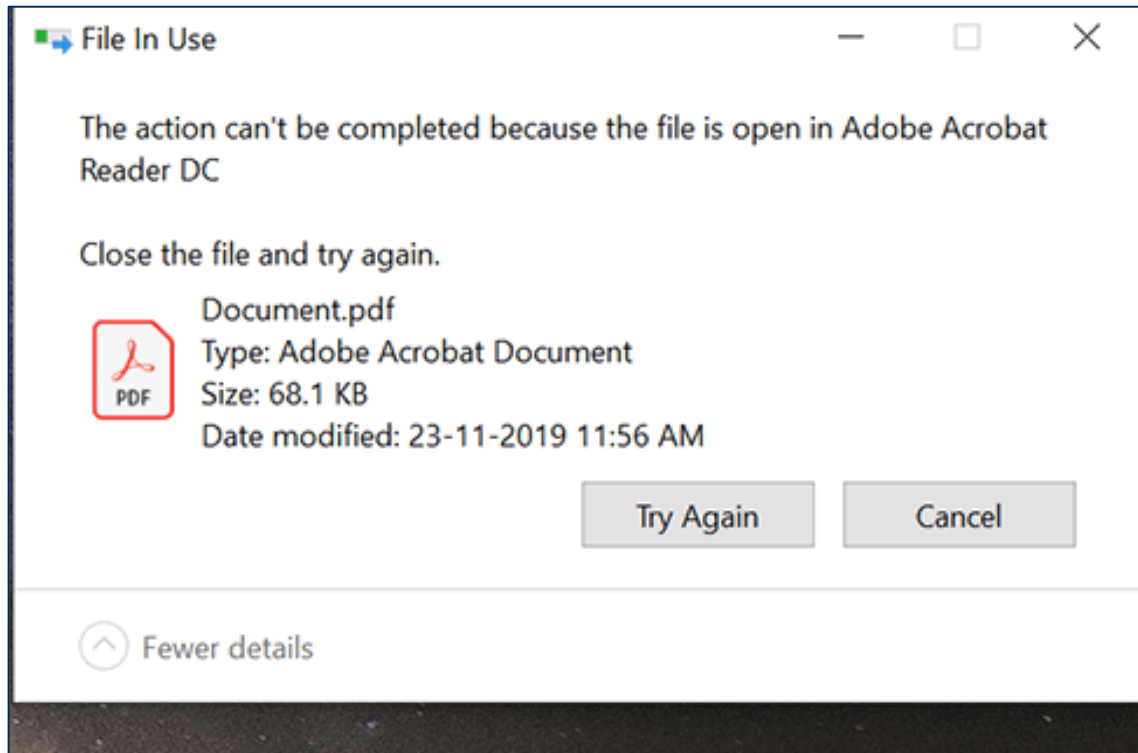




Today's topics

- Concurrency problem
 - Activities with example, initial contents on locks (more to come in the next couple of weeks)
- Deadlock
 - Activities, combine with other topics
- Week 5's quiz discussion (if you have any question)

Concurrency problem



Concurrent transactions can cause issues in Database – need concurrency control

Concurrency Problems

Transaction 1 $\text{balance} = \text{balance} + 50;$	Transaction 2 $\text{balance} = \text{balance} - 80;$	Account balance = 100;
--	--	------------------------

Both transactions are running concurrently. What are the possible outcomes if no concurrency control in place? Which ones are correct sequence of actions?

Write by a transaction gets
lost/overwritten

a) $\text{balance} == 150$

Sequence of actions

T1: Reads $\text{balance} == 100$

T2: Reads $\text{balance} == 100$

T2: Writes $\text{balance} == 100-80$

T1: Writes $\text{balance} == 100+50$

b) $\text{balance} == 20$

Sequence of actions

T1: Reads $\text{balance} == 100$

T2: Reads $\text{balance} == 100$

T1: Writes $\text{balance} == 100+50$

T2: Writes $\text{balance} == 100-80$

c) $\text{balance} == 70;$

Sequence of actions

T1: Reads $\text{balance} == 100$

T1: Writes $\text{balance} == 100+50$

T2: Reads $\text{balance} == 150$

T2: Writes $\text{balance} == 150-80$

d) $\text{balance} == 70;$

Sequence of actions

T2: Reads $\text{balance} == 100$

T2: Writes $\text{balance} == 100-80$

T1: Reads $\text{balance} == 20$

T1: Writes $\text{balance} == 20+50$

Concurrency Problems

Transaction 1

balance = balance + 50;

Transaction 2

balance = balance - 110;

Account balance = 100;

Both transactions are running concurrently. What are the possible outcomes if no concurrency control in place? Which ones are correct sequence of actions?
(assume overdrawing not allowed)

c) balance == 40;

Sequence of actions

T1: Reads balance == 100

T1: Writes balance == 100+50

T2: Reads balance == 150

T2: Writes balance == 150-110

d) balance == 150;

Sequence of actions

T2: Reads balance == 100

T2: Writes balance == 100-110

(message on overdrawing denied)

T1: Reads balance == 100

T1: Writes balance == 100+50

Try withdrawing money
after some time



Poll - [Pollev.com/farhanachoud585](https://pollev.com/farhanachoud585)



Concurrency Control

- To resolve conflicts
- To preserve database consistency

Different lock implementations for concurrency control

- Dekker's algorithm and through OS interruption
- **Spin locks** – most commonly used

Two implementations of spin lock (test&set, compare&swap)

T1

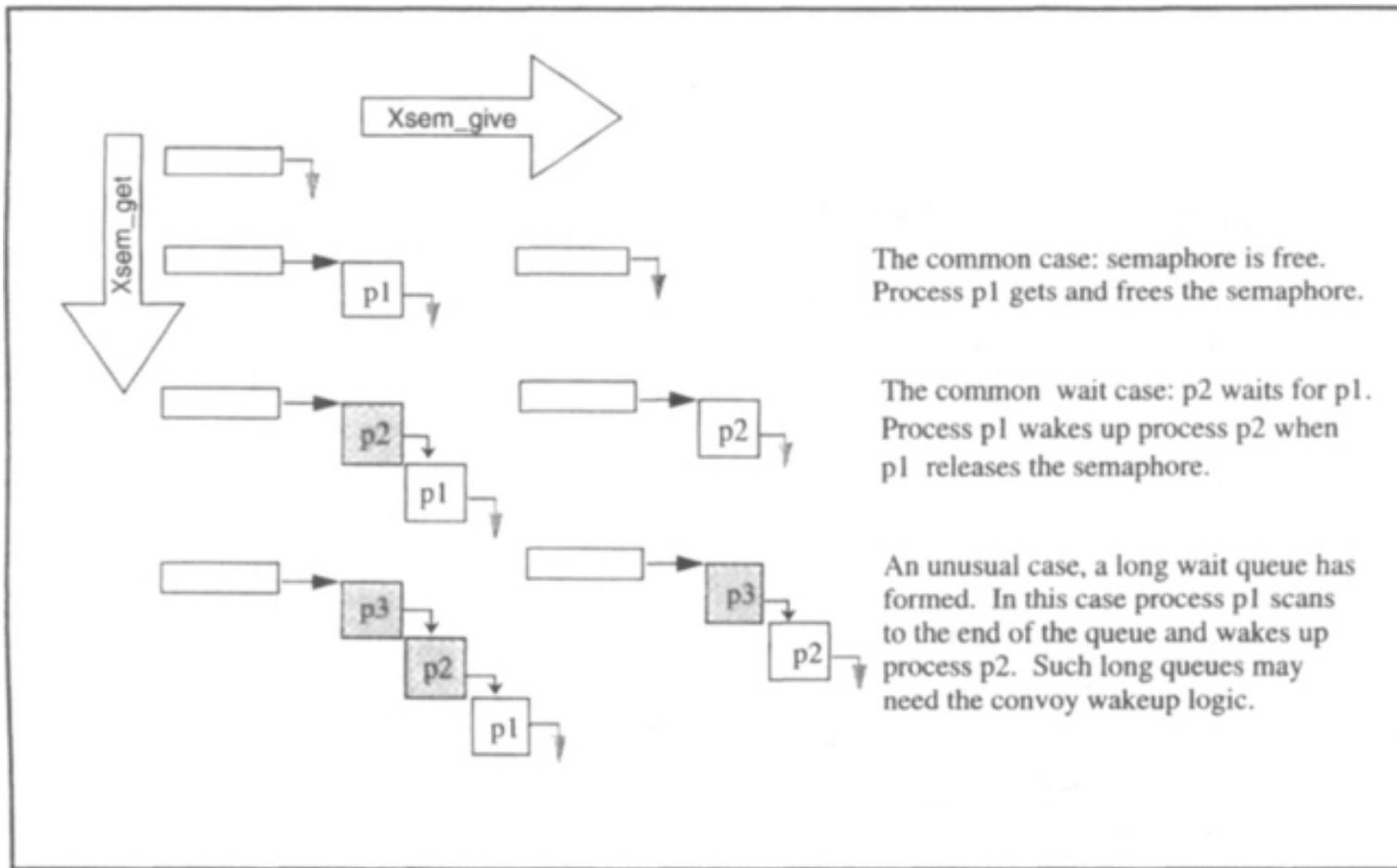
```
/*acquire lock*/  
while (!testAndSet( &lock );  
    /*Xlock granted*/  
//exclusive access for T1;  
counter = counter+1;  
/* release lock*/  
lock = 1;
```

T2

```
/*acquire lock*/  
while (!testAndSet( &lock );  
    /*Xlock granted*/  
//exclusive access for T2;  
counter = counter+1;  
/* release lock*/  
lock = 1;
```

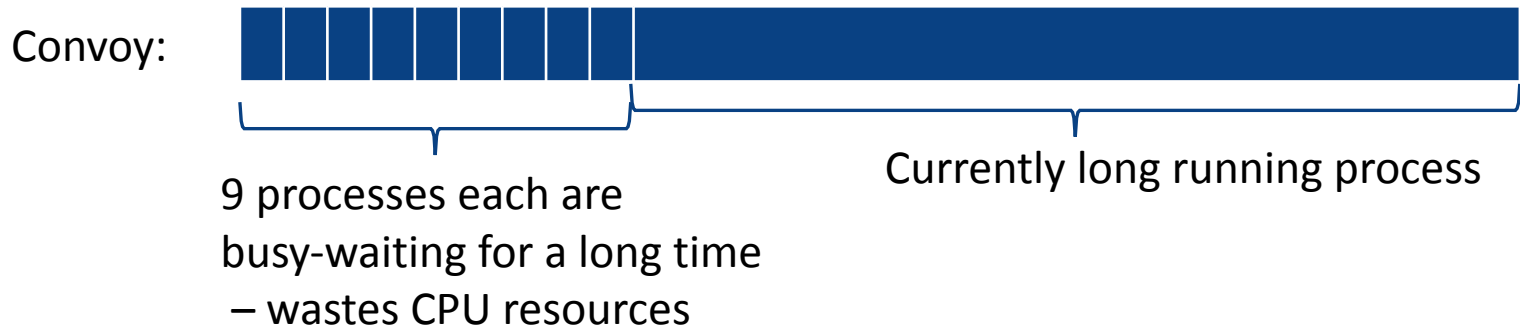
Concurrency Control

General purpose locks - semaphores



Concurrency Control

Convoy avoiding semaphore – some processes may have to wait behind some long running processes, which forms a long queue of waiting processing. To avoid a long list of waiting processes (called convoy), simply free the semaphore and then wake up every process in the list after usage so that each of those processes now get an equal chance to acquire semaphore.



Concurrency Control

Convoy avoiding semaphore – some processes may have to wait behind some long running processes, which forms a long queue of waiting processing. To avoid a long list of waiting processes (called convoy), simply free the semaphore and then wake up every process in the list after usage so that each of those processes now get an equal chance to acquire semaphore.

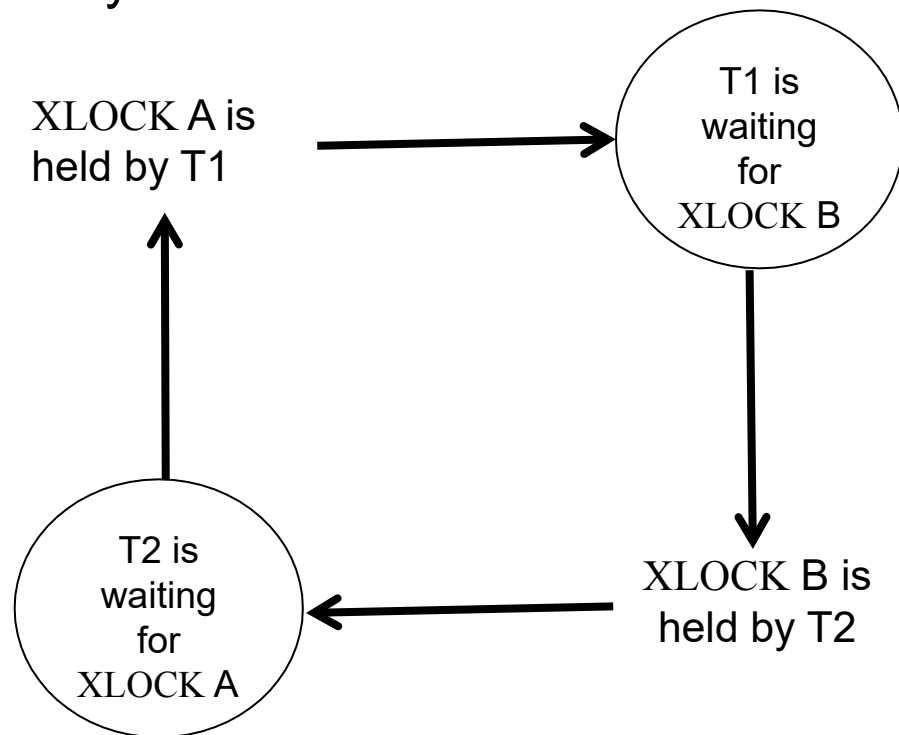
In a database, the table 'student_records' is very frequently accessed by many concurrent transactions, for both reading and writing. Some of the transactions require a long time to run. What type of semaphore should be used for this database table?



Poll - [Pollev.com/farhanachoud585](https://pollev.com/farhanachoud585)

Deadlocks ...

Deadlocks are rare, however, they do occur and the database has to deal with them when they occur



Can proper indexing reduce the changes of a deadlock?



Deadlock avoidance/mitigation

- Pre-declare all necessary resources and allocate in a single request.
- Periodically check the resource dependency graph for cycles. If a cycle exists - rollback (i.e., terminate) one or more transaction to eliminate cycles (deadlocks). The chosen transactions should be cheap (e.g., they have not consumed too many resources).
- Allow waiting for a maximum time on a lock then force Rollback. Many successful systems (IBM, Tandem) have chosen this approach.
- Many distributed database systems maintain only local dependency graphs and use time outs for global deadlocks.



Deadlocks

Many systems employ a simple strategy to handle deadlocks - they just allow a transaction to wait for certain maximum time on a lock, and then force it to rollback. Why is this strategy used by most systems? What are the benefits of this strategy over the other possible strategies to handle deadlocks?



Concurrency problem

Multiple concurrently running transactions may cause conflicts

- **Still we try to allow** concurrent runs as much as possible for a better performance, while avoiding conflicts as much as possible

Topics for next two weeks

Core Concepts of Database management system

